

The Build Process

Where We Left Off

We have been typing commands like:

```
gcc -g -O0 main.c -o main  
cargo build
```

and getting a program that runs

Today: what happens in between

Source Code Is Not a Program

The CPU runs **machine instructions**

Source code is **text** written for humans

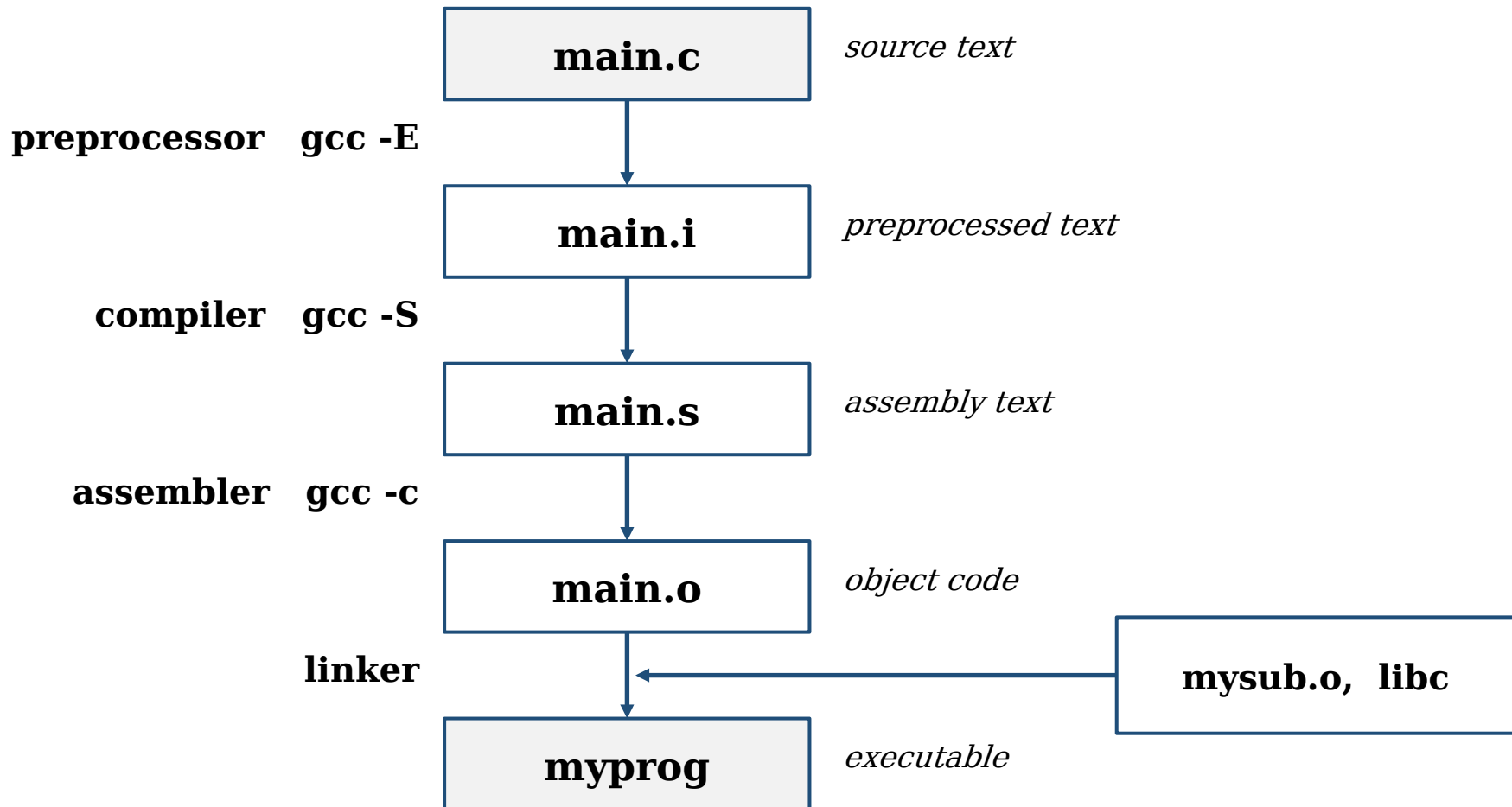
The **build** translates the text into an **executable** the operating system can load

Many steps - and many ways to go wrong

Building C: Four Stages

1. **Preprocess** handle the # lines
2. **Compile** C to assembly language
3. **Assemble** assembly to object code
4. **Link** object code to executable

The C Pipeline



Stage 1: Preprocess

Handles every line that begins with **#**

<code>#include</code>	paste in another file
<code>#define</code>	substitute text
<code>#if #ifdef</code>	keep or drop code

`gcc -E main.c -o main.i` stop after this
stage

Pure **text** in, text out

What the Preprocessor Makes

hello.c 5 lines

```
#include <stdio.h>
int main ( void ) {
    printf ( "hello\n" ) ;
}
```

hello.i **hundreds** of lines
all of stdio.h, pasted in

Stage 2: Compile

Translate the C in `main.i` into **assembly**

This is where the compiler:

- checks types and syntax - **errors**

- spots suspicious code - **warnings**

- optimizes (`-O0`, `-O2`)

`gcc -S main.c` produces `main.s`

What Assembly Looks Like

```
int square ( int x ) { return x * x ; }
```

gcc -O2 -S on x86-64:

```
square:  
    imull    %edi, %edi  
    movl     %edi, %eax  
    ret
```

One line of C, three machine instructions

Stage 3: Assemble

Assembly text to **machine code**

`gcc -c main.c` produces `main.o`

An **object file** holds:
the machine code (text)
the data
a **symbol table**

Object Files Are Incomplete

```
nm main.o
```

```
  T main          defined here
```

```
  U my_sub        undefined - somewhere else
```

```
  U printf        undefined - in the C library
```

Calls to other files point to a **placeholder**

The final addresses are not known yet

Stage 4: Link

Combine all the object files and libraries

Match every **U** with a **T** somewhere

Assign every symbol its **final address**

```
gcc main.o mysub.o -o myprog
```

The result: one executable file

Link Errors

undefined reference to 'my_sub'
you forgot to link mysub.o
or misspelled the name

multiple definition of 'countG'
two files **define** it - one should say extern

The code compiled fine - the **pieces** do not fit

Compile Errors vs Link Errors

	Compile error	Link error
Found by	the compiler	the linker
Looks at	one source file	all object files
Message has	file and line number	a symbol name
Typical cause	syntax, types	missing or duplicate definition
Fix	edit that line	add a file, library, or extern

One Command, Four Stages

```
gcc -Wall -g main.c mysub.c -o myprog
```

Preprocesses, compiles, and assembles each file,
then links them - intermediate files are deleted

Keep them to look at:

```
gcc -save-temps main.c -o main
```

You get main.i, main.s, main.o

gcc Flags You Will Use

-Wall -Wextra	turn on warnings
-g	include debug information
-O0 -O2	optimization level
-std=c17	language version
-c	compile only, no link
-o name	name the output
-I dir	where to find headers
-L dir -lname	where and which libraries

Libraries

A **library** is a collection of object files

Static: .a (.lib)
 copied **into** your executable

Shared: .so .dylib .dll
loaded when the program **starts**

gcc main.c -lm link the math library

Headers and Libraries Are Different

Header (.h) gives **declarations**
used by the **compiler**

Library (.a .so) gives **definitions**
used by the **linker**

`#include <math.h>` without `-lm`
compiles fine, then fails to **link**

Separate Compilation

Each .c file compiles **on its own**

Change one file:
 recompile only that file
 relink everything

A large project has **thousands** of files
Rebuilding all of them takes a long time

But Which Files Changed?

Edit mysub.c only mysub.o is stale

Edit mysub.h every file that **includes**
it is stale

Keeping track by hand is error prone

Forget one, and you link **old** code

make

A **Makefile** lists rules:

```
target : dependencies  
<TAB>  command to build the target
```

make rebuilds a target only if a dependency is **newer** than the target

The command line **must** start with a TAB

A Makefile

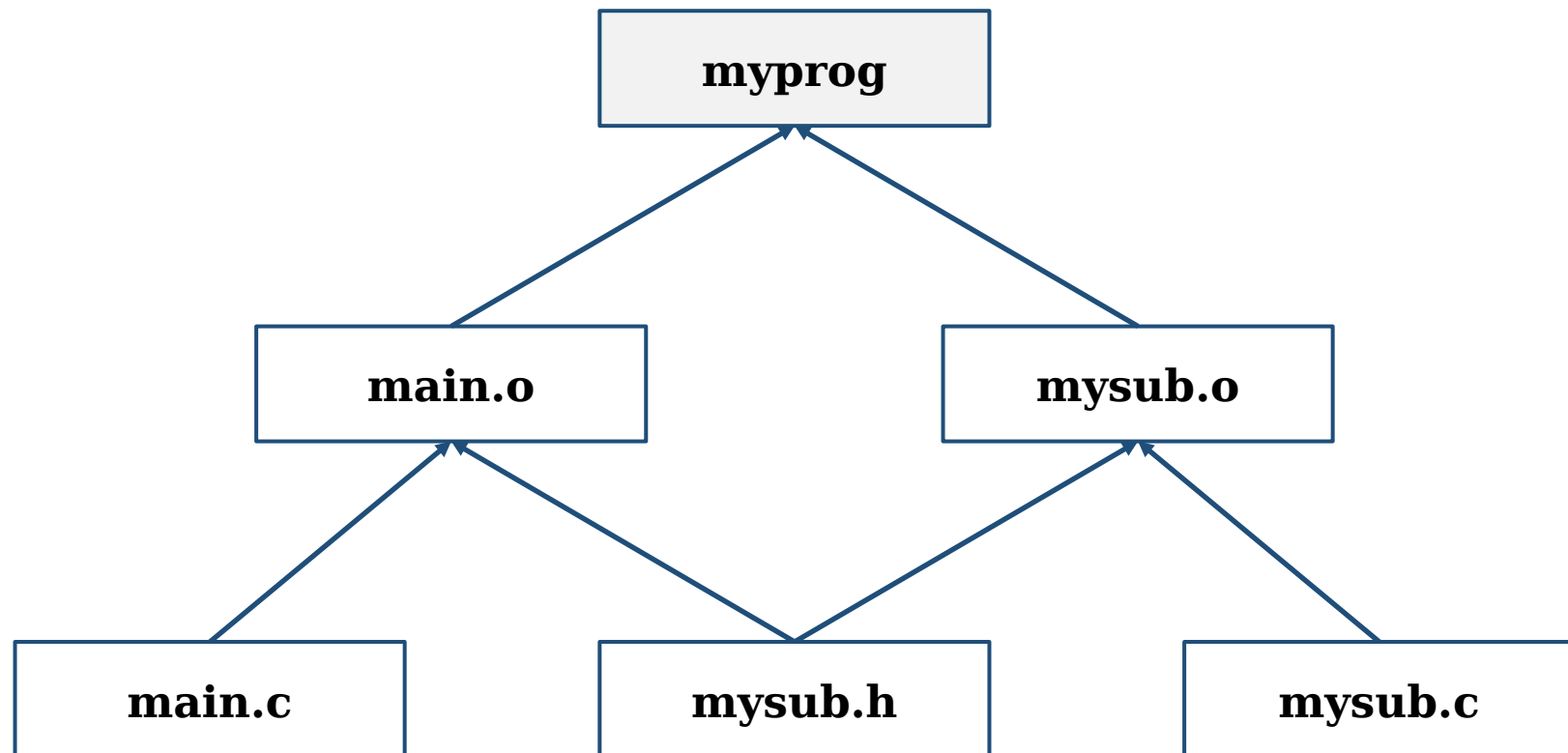
```
CFLAGS = -Wall -g
```

```
myprog : main.o mysub.o  
    gcc main.o mysub.o -o myprog
```

```
main.o : main.c mysub.h  
    gcc $(CFLAGS) -c main.c
```

```
mysub.o : mysub.c mysub.h  
    gcc $(CFLAGS) -c mysub.c
```

The Dependency Graph



Arrows point from a dependency to what is built from it

CMake

Writes the Makefiles **for** you

CMakeLists.txt:

```
cmake_minimum_required ( VERSION 3.16 )  
project ( myprog C )  
add_executable ( myprog main.c mysub.c )
```

cmake -B build

cmake --build build

The Missing Piece

How do you use code **someone else** wrote?

C and C++ have **no standard** answer:

- install a system package

- copy the source into your project

- tools such as vcpkg or Conan

Versions, flags, and platforms must all match

Building C++

The **same** four stages

```
g++ -std=c++20 -Wall -g main.cpp sample.cpp  
-o prog
```

Use `g++` or `clang++`, not `gcc`, to link
so the C++ standard library is included

Name Mangling

C++ allows overloading - many `to_psi` functions

So symbol names encode the parameter types:

```
float to_psi ( unsigned )    _Z6to_psij
```

`nm -C` shows readable names

`extern "C"` turns mangling off, to call C code

Why C++ Builds Are Slow

Templates live in **headers**

Every file that includes them compiles them again

`#include <iostream>` alone adds tens of thousands of lines to each file

C++20 **modules** begin to fix this

Building Rust

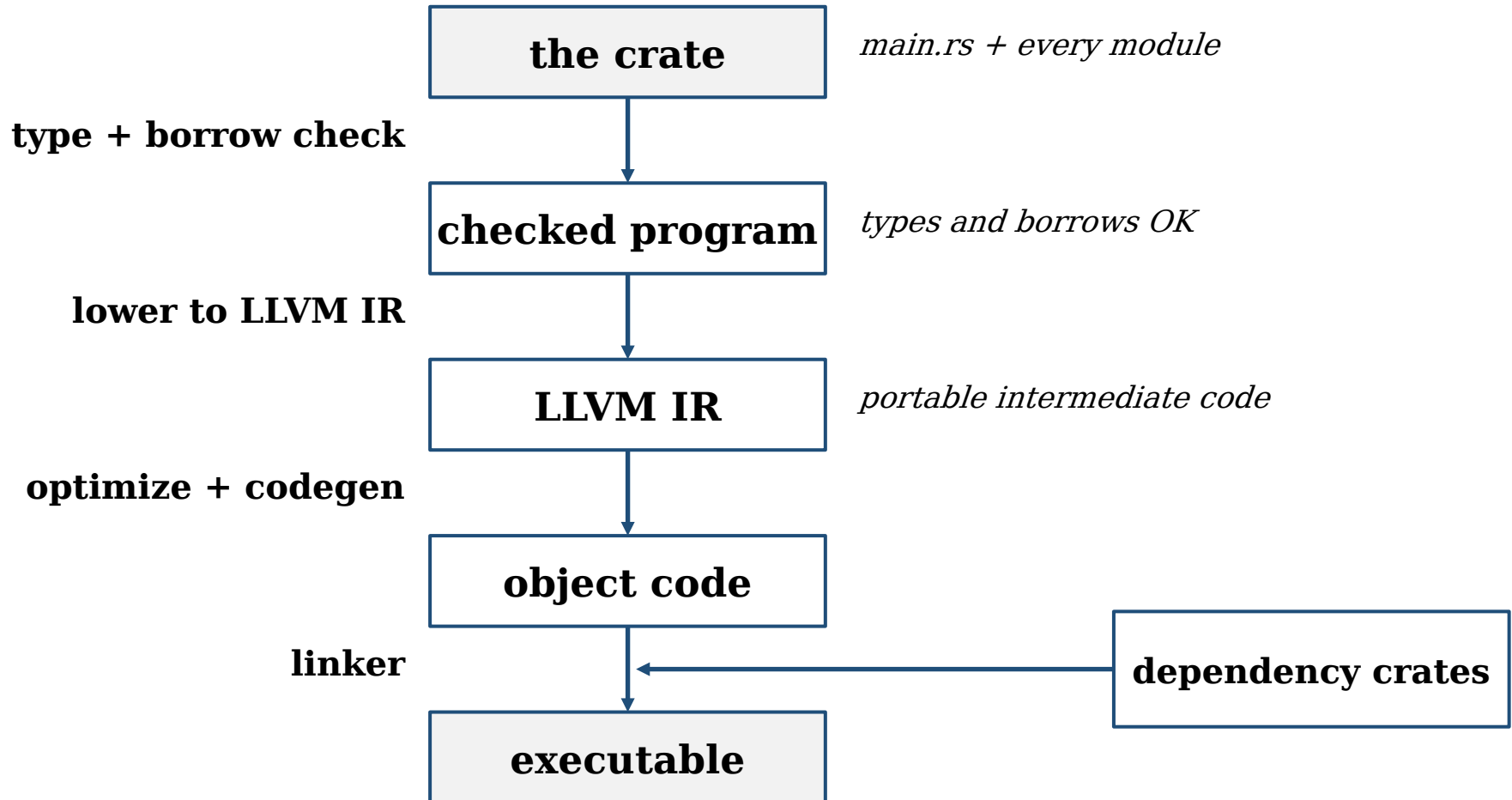
No preprocessor. No header files.

The compiler, **rustc**, compiles a whole **crate** at once - not one file at a time

Linking still happens, using the system linker

`rustc main.rs` makes `main`

The Rust Pipeline



C and Rust Compared

	C	Rust
Unit of compilation	one .c file	one crate
Declarations	header files	the compiler sees all
Text inclusion	#include	none
Build description	Makefile, CMake	Cargo.toml
Other people's code	install by hand	crates.io + Cargo

What Is a Crate?

The **unit of compilation** in Rust

One crate = one run of rustc

Produces either an **executable**
or a **library** other crates can use

A crate may be one file or hundreds

Binary and Library Crates

Binary crate

- has a main function
- builds an **executable**

Library crate

- no main
- builds a `.rlib` for other crates to use

Like a C program versus a C library

The Crate Root

The file `rustc` starts from:

<code>src/main.rs</code>	binary crate
<code>src/lib.rs</code>	library crate

Every other file joins the crate through a **mod** line in the root or its children

Modules Inside a Crate

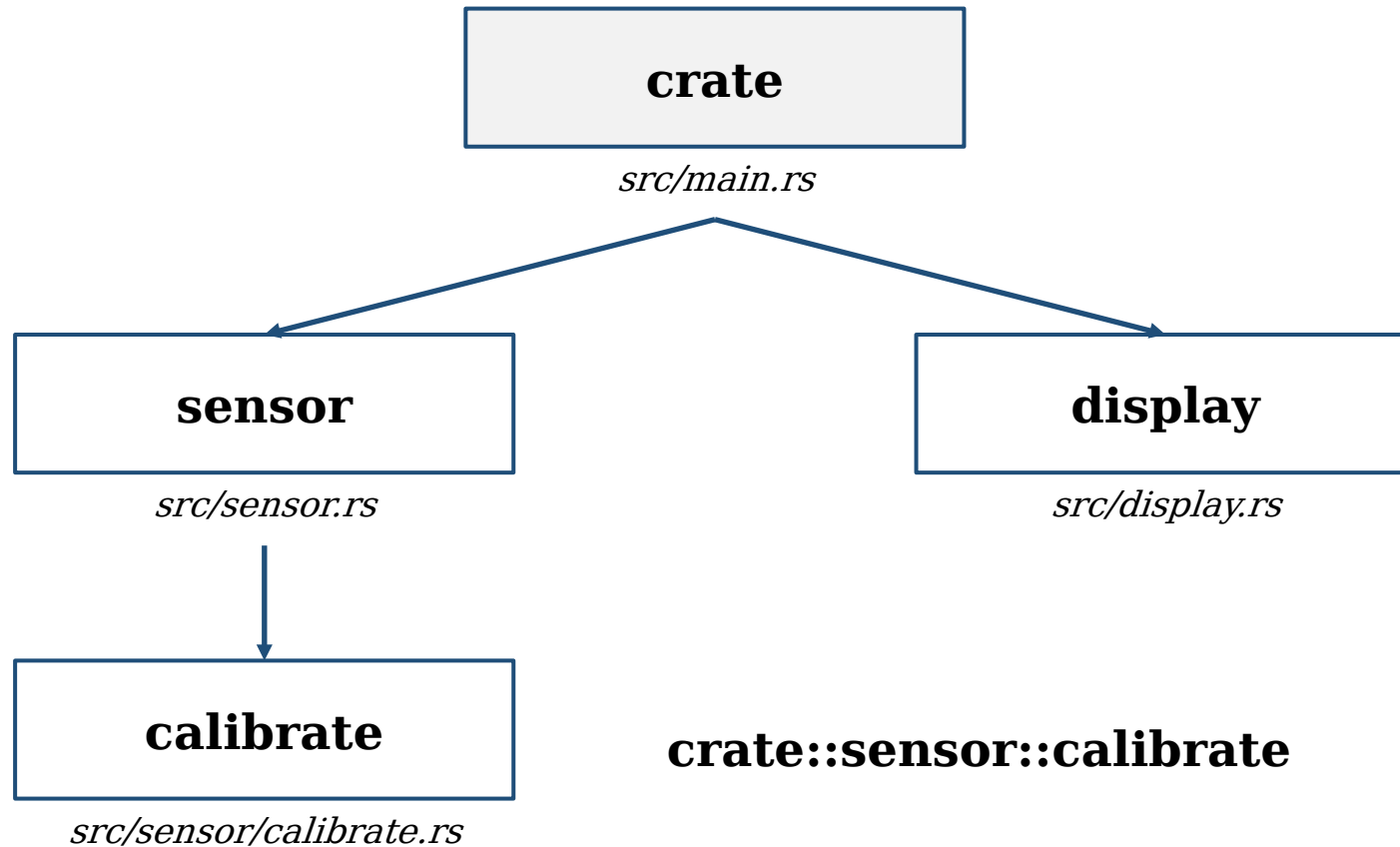
src/main.rs

```
mod sensor ;           loads src/sensor.rs
mod display ;          loads src/display.rs
```

src/sensor.rs

```
pub mod calibrate ; loads src/sensor/calibrate.rs
pub fn adc_to_psi ( counts : u32 ) -> f32 { ... }
```

The Module Tree



Paths to Items

<code>crate::sensor::adc_to_psi</code>	from the crate root
<code>self::helper</code>	this module
<code>super::adc_to_psi</code>	the parent module

`use` shortens a path:

```
use crate::sensor::adc_to_psi ;  
let psi = adc_to_psi ( 1456 ) ;
```

Visibility in a Crate

<code>fn helper ()</code>	private to its module
<code>pub fn adc_to_psi ()</code>	visible to parent and
<code>beyond</code>	
<code>pub(crate) fn tool ()</code>	anywhere in this crate

Only **pub** items can be used by other crates

The crate is the boundary of a library's API

Packages

A **package** is a Cargo.toml plus its crates

At most **one** library crate

Any number of binary crates:

- src/main.rs

- src/bin/calibrate.rs

- src/bin/log_dump.rs

A Package on Disk

oil_gauge/	
Cargo.toml	the manifest
Cargo.lock	exact dependency versions
src/	
main.rs	binary crate root
lib.rs	library crate root
sensor.rs	a module
tests/	integration tests
target/	all build output

Cargo

Rust's **build tool** and **package manager**

One tool to:

- create a package

- build and run it

- test it

- fetch and build its dependencies

Replaces make, CMake, and hand-installed libraries

cargo new

```
cargo new oil_gauge
```

Creates:

- Cargo.toml

- src/main.rs with a hello world

- a **git** repository and .gitignore

```
cargo new --lib gauge_core    for a library
```

Cargo.toml

```
[package]
name = "oil_gauge"
version = "0.1.0"
edition = "2024"
```

```
[dependencies]
```

The **manifest**: what the package is,
and what it needs

Editions

An **edition** fixes the language rules for a crate:
2015, 2018, 2021, 2024

New editions can change syntax without
breaking old code

Crates of **different** editions link together
without any problem

Everyday Cargo Commands

cargo build
cargo run
cargo check
cargo test
cargo build --release
target/release
cargo clean
cargo doc --open
crate

compile into target/debug
build if needed, then run
type check only - fast
build and run the tests
optimized, into

delete target/
documentation for every

Debug and Release Profiles

debug (cargo build)
no optimization, full debug info
overflow **panics**

release (cargo build --release)
fully optimized, no debug info
overflow **wraps**

cargo check

Runs the **type** and **borrow** checks
but generates no machine code

Much faster than `cargo build`

Use it constantly while writing code
Build when you are ready to run

Using Someone Else's Crate

crates.io is the public registry of Rust crates

Add a dependency:

```
cargo add rand
```

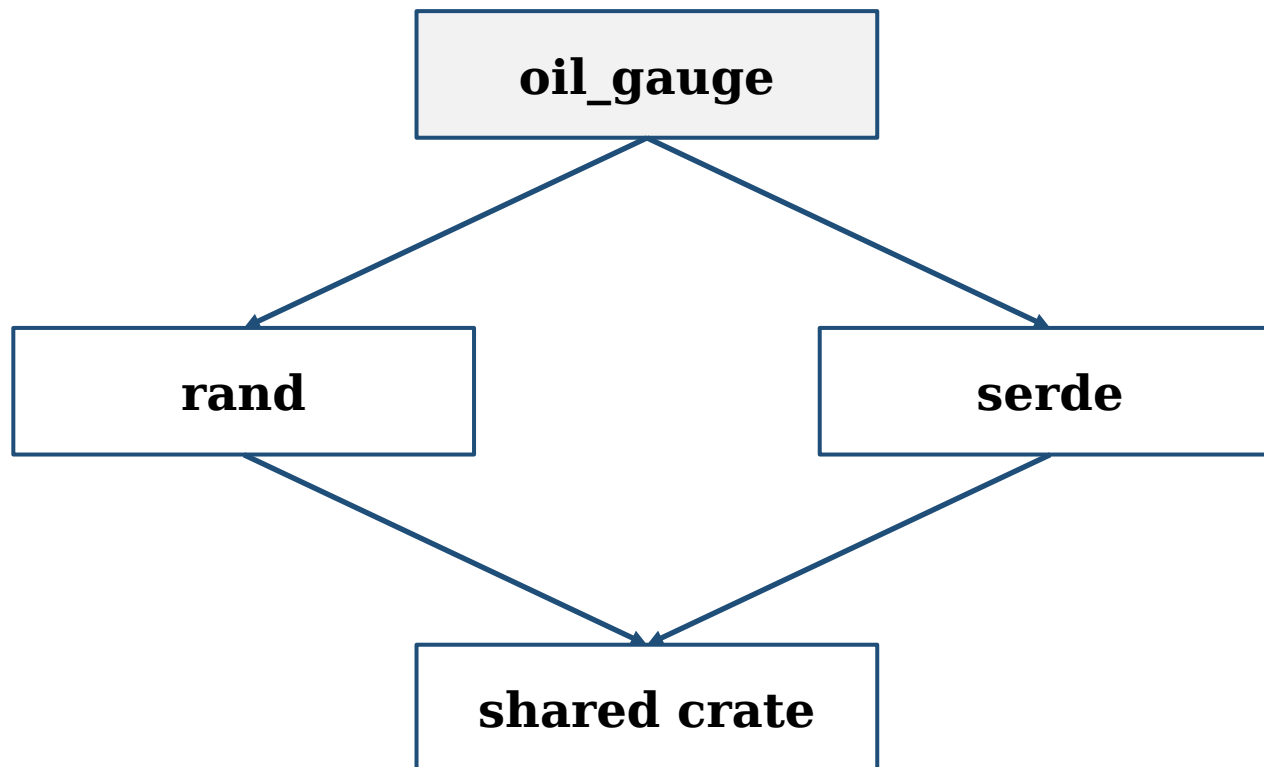
or edit Cargo.toml by hand:

```
[dependencies]  
rand = "0.9"
```

What Cargo Does With It

1. **Download** the crate's source
2. Do the same for **its** dependencies
3. **Compile** each one as its own crate
4. **Link** them all into your program

The Crate Dependency Graph



Arrows point to dependencies. The shared crate is built only once

Version Requirements

Semantic versioning: MAJOR . MINOR . PATCH

rand = "0.9"	any compatible 0.9.x
serde = "1.0"	1.0.0 up to, not including,
2.0.0	
foo = "=1.2.3"	exactly 1.2.3

A new **MAJOR** version may break your code
Cargo will not pick one without being told

Cargo.lock

Records the **exact** version of every crate that was chosen

Everyone who builds gets the **same** code

Commit it to git

cargo update
versions

move to newer compatible

Using a Dependency

```
[dependencies]  
rand = "0.9"
```

```
fn main ( ) {  
    let noise : f32 = rand::random ( ) ;  
    println! ( "simulated psi = {:.1}", 50.0 + noise ) ;  
}
```

The crate name is the first part of the path

How Crates Speed Up the Build

Each crate compiles **once** and is cached in target/

Change your code: only **your** crate rebuilds

Dependencies are reused until their version or settings change

Incremental Compilation

Inside **your** crate, rustc also remembers what it did last time

Only the parts affected by your edit are recompiled

On by default in debug builds

Parallel Builds

Crates that do not depend on each other compile **at the same time**

Cargo uses every core in the machine

A project split into several crates builds faster than one giant crate

No Header Bookkeeping

The compiler sees the whole crate, so there are:

- no prototypes to keep in sync
- no include guards
- no hand-written dependency lists
- no stale object files

A whole class of C build bugs cannot happen

Workspaces

Several crates, one Cargo.lock, one target/

Cargo.toml at the top:

```
[workspace]
```

```
members = [ "gauge_core", "gauge_app" ]
```

Split a big project into crates
for faster rebuilds and clearer boundaries

Building for a Microcontroller

The same crate can target another CPU:

```
rustup target add thumbv7em-none-eabihf  
cargo build --target thumbv7em-none-eabihf
```

Cross compiling is built in

ESP32 chips use the `espup` toolchain installer

The Build and the Debugger

The debugger needs the build's **debug info**

C, C++: `-g -O0`

Rust: `cargo build ( debug profile )`

Debugging a release build?

`[profile.release]`

`debug = true`

The Two Builds Side by Side

Step	C / C++	Rust
Compile	gcc -c per file	rustc per crate
Link	gcc *.o -l...	automatic
Rebuild rules	Makefile, CMake	Cargo
Libraries	install, -I -L -l	Cargo.toml
Lock versions	by hand	Cargo.lock
Debug build	-g -O0	cargo build
Optimized build	-O2	cargo build --release

What We Can Now Do

Name the **four** stages of a C build

Tell a **compile** error from a **link** error

Read a **Makefile** and use **CMake**

Organize Rust code into **crates** and **modules**

Use **Cargo** to build, test, and add dependencies

Let's practice

Exercise: Using **buildpractice**